SMIT | AI & DATA SCIENCE

PROJECT REPORT

AI Bug Report Analyzer

An intelligent multi-step workflow built with LangGraph, Google Gemini and FastAPI

SUBMITTED BY	**Anushay Ayat**
COURSE	**AI and Data Science**
BATCH	**12**
INSTITUTE	**SMIT**
SUBMISSION DATE	**22 / 10 / 2026**

Assignment: AI Bug Report Analyzer (LangGraph)

Table of Contents

1. Introduction.....4
2. Objectives and Scope.....5
3. Requirements Analysis.....6
4. System Design.....7
5. Backend Implementation.....9
6. Frontend Implementation.....11
7. Application Screenshots.....12
8. Testing and Validation.....15
9. Sample Input and Output.....16
10. Challenges and Solutions.....17
11. Limitations and Future Work.....18
12. Conclusion.....19
References.....19

Abstract

Software teams receive bug reports written in many different styles: some are long and vague, others are a single sentence with no context. Triaging them manually is slow and inconsistent. This project presents the AI Bug Report Analyzer, a full-stack web application that converts a plain-English bug description into a structured, developer-ready bug report.

The core of the system is a four-node LangGraph workflow. Node 1 extracts the bug title, the affected feature and a concise summary. Node 2 classifies the severity as Critical, High, Medium or Low and explains the decision. Node 3 proposes plausible technical root causes. Node 4 assembles everything, together with steps to reproduce and recommended next steps, into a clean Markdown report. Each node calls Google Gemini through LangChain using Pydantic structured output so that the results are validated and predictable.

The backend is implemented with FastAPI and exposes both a standard JSON endpoint and a Server-Sent Events endpoint that streams progress node by node. The frontend is a responsive, professionally designed website that shows live workflow progress, colour-coded severity, tabbed report views, copy and download options and a local analysis history. An offline rule-based engine guarantees that the application keeps working when no API key is configured. The project is accompanied by automated tests, API documentation and a detailed README.

Keywords: LangGraph, Gemini, FastAPI, bug triage, severity classification, root cause analysis, structured output.

1. Introduction

1.1 Background

Bug tracking is a daily activity in every software team. The quality of a bug report strongly affects how quickly a defect is understood and fixed. Reports that lack a clear title, an identified feature, a severity and a first set of hypotheses force developers to ask follow-up questions and slow the whole process down.

1.2 Problem Statement

Users and testers describe bugs in natural language. There is no consistent way to turn this free text into a uniform, structured artefact that contains the information developers need. The assignment asks for an AI assistant that takes a bug description and produces such a structured report using Gemini and LangGraph.

1.3 Proposed Solution

The proposed solution decomposes the task into four small, specialised steps arranged as a LangGraph state graph. Splitting the work this way produces more focused prompts, makes each step independently testable and allows the pipeline to grow easily. The result is delivered through a modern web interface so that non-technical users can use it without any setup.

2. Objectives and Scope

2.1 Objectives

- Accept a bug description written in natural language.
- Extract the bug title, affected feature and issue summary (Node 1).
- Classify severity as Critical, High, Medium or Low (Node 2).
- Suggest possible root causes (Node 3).
- Generate a clean, readable structured bug report (Node 4).
- Provide a professional frontend, a well-organised backend and clear documentation.

2.2 Scope

The project covers the complete workflow from text input to final report, a REST and streaming API, a single-page web interface, automated tests and documentation. Persistent storage, user accounts and direct integration with issue trackers are outside the current scope and are listed as future work.

3. Requirements Analysis

3.1 Functional Requirements

ID	Requirement
FR-1	The user can enter a bug description (10 to 4000 characters).
FR-2	The system runs a four-node LangGraph workflow in a fixed order.
FR-3	The system returns title, feature, summary, severity, root causes, steps and next steps.
FR-4	The user sees live progress for every node.
FR-5	The user can copy the report and download it as Markdown or JSON.
FR-6	Recent analyses are available in a history list.

3.2 Non-Functional Requirements

- **Reliability:** invalid input is rejected with clear errors; LLM failures fall back to an offline engine.
- **Usability:** responsive layout, clear colour coding and one-click actions.
- **Maintainability:** modular folder structure, prompts separated from logic, typed state and schemas.
- **Performance:** progress is streamed so users get feedback while the workflow runs.

3.3 Technology Choices

Layer	Technology	Reason
Orchestration	LangGraph	Explicit, typed state machine for multi-step LLM flows
LLM	Google Gemini	Strong structured-output support and a free tier
Backend	FastAPI + Uvicorn	Fast, async-ready, automatic OpenAPI docs
Validation	Pydantic v2	Schemas for LLM output and API payloads
Frontend	HTML, CSS, JavaScript	No build step, easy to deploy and review

4. System Design

4.1 High-Level Architecture

The browser sends the bug description to the FastAPI server. The server invokes the compiled LangGraph workflow, which uses the LLM service layer to call Gemini. Results are returned as JSON or streamed as Server-Sent Events. FastAPI also serves the static frontend, so the whole application runs from a single command.

```
Browser (HTML/CSS/JS)
   |  POST /api/analyze/stream
   v
FastAPI (routes.py)
   |  build_graph().stream()
   v
LangGraph StateGraph
   START -> bug_analysis -> severity_detection -> root_cause -> format_report ->
END
                 |  each node calls
                 v
LLM service (llm.py) --> Gemini (structured output)
                 '--> Demo engine (fallback)
```

4.2 Workflow Design

Node	Input fields	Output fields	Purpose
bug_analysis	bug_description	title, affected_feature, summary	Understand and summarise the bug
severity_detection	title, feature, summary	severity, severity_reason	Classify impact
root_cause	all previous fields	root_causes	Suggest likely causes
format_report	all previous fields	steps_to_reproduce, suggested_next_steps, final_report	Build the final report

4.3 Shared State

All nodes communicate through a single typed dictionary. A node never calls another node; it only reads from the state and returns the keys it adds. LangGraph merges the returned values, which keeps nodes independent and easy to test.

```
class BugState(TypedDict, total=False):
    bug_description: str
    title: str; affected_feature: str; summary: str
    severity: str; severity_reason: str
    root_causes: List[str]
    steps_to_reproduce: List[str]; suggested_next_steps: List[str]
    final_report: str
```

4.4 Severity Model

Level	Definition
Critical	Crash, data loss, security vulnerability or payments completely blocked.
High	A major feature is broken and there is no reasonable workaround.
Medium	A feature is degraded or inconsistent but a workaround exists.
Low	Cosmetic issue or minor inconvenience.

5. Backend Implementation

5.1 Folder Structure

```
backend/app/
  main.py            FastAPI app, CORS, static frontend
  config.py          environment settings
  schemas.py         Pydantic models
  api/routes.py      /api/health, /samples, /analyze, /analyze/stream
  graph/state.py     BugState
  graph/prompts.py   prompt templates
  graph/nodes.py     the four nodes
  graph/workflow.py  graph wiring
  services/llm.py    Gemini + fallback
  services/demo_engine.py  offline engine
```

5.2 LangGraph Workflow

The graph is created with StateGraph(BugState). Each node is registered by name and connected with sequential edges from START to END. The compiled graph is cached so it is built only once.

```
g = StateGraph(BugState)
g.add_node("bug_analysis", bug_analysis_node)
g.add_node("severity_detection", severity_detection_node)
g.add_node("root_cause", root_cause_node)
g.add_node("format_report", format_report_node)
g.add_edge(START, "bug_analysis")
g.add_edge("bug_analysis", "severity_detection")
g.add_edge("severity_detection", "root_cause")
g.add_edge("root_cause", "format_report")
g.add_edge("format_report", END)
return g.compile()
```

5.3 Gemini Integration and Structured Output

The LLM service wraps ChatGoogleGenerativeAI and calls with_structured_output() with a Pydantic schema for every node. Gemini therefore returns validated objects (BugAnalysis, SeverityResult, RootCauses, ReportExtras) instead of free text that would need fragile parsing. The model name and API key are read from the .env file.

5.4 Prompt Design

Prompts are stored separately in prompts.py. Each prompt gives the model a role (for example senior QA engineer), a precise task, the allowed output values and only the context fields that the step needs. The severity prompt contains explicit definitions of the four levels to keep classification consistent.

5.5 Fallback Engine

If no valid API key is configured, or a Gemini call fails because of network or quota problems, the service falls back to a rule-based engine. It detects the feature from keywords, assigns severity from

impact keywords and returns curated root-cause suggestions. The navbar badge in the UI shows which engine is active. This engine is a heuristic and is not a replacement for the LLM; it exists for offline demonstrations and graceful degradation.

5.6 REST and Streaming API

Method	Endpoint	Description
GET	/api/health	Status, version and active engine
GET	/api/samples	Sample bug descriptions
POST	/api/analyze	Run the workflow and return the report
POST	/api/analyze/stream	SSE: one event per node, then the final report

The streaming endpoint uses graph.stream(stream_mode="updates") and emits an event after each node completes. The frontend consumes the stream with fetch and a ReadableStream reader.

6. Frontend Implementation

The frontend is written in plain HTML, CSS and JavaScript so that it needs no build tools. It is served directly by FastAPI, which also avoids cross-origin issues.

6.1 Page Sections

- **Hero:** title, short explanation and call-to-action buttons.
- **Analyzer workspace:** input area with sample bug chips, character counter, workflow progress tracker and result panel.
- **Workflow section:** visual explanation of the four nodes.
- **Features and History:** capability cards and the list of recent analyses.

6.2 User Experience Features

- Live stepper: each node changes from Waiting to Running to Done as events arrive.
- Severity badges coloured red, orange, yellow and green.
- Report, Markdown and JSON tabs for different audiences.
- Copy to clipboard, download as .md and download as .json.
- History stored in localStorage; selecting an item re-opens the report.
- Responsive layout that collapses to a single column on tablets and phones.

6.3 Design System

The interface uses a dark graphite palette with a teal accent, defined as CSS custom properties. Typography relies on a system font stack with a monospace face for technical labels. Spacing, border radius and colours are reused consistently across components.

7. Application Screenshots

The figures below show the running application. They were captured automatically with Playwright using the offline engine.

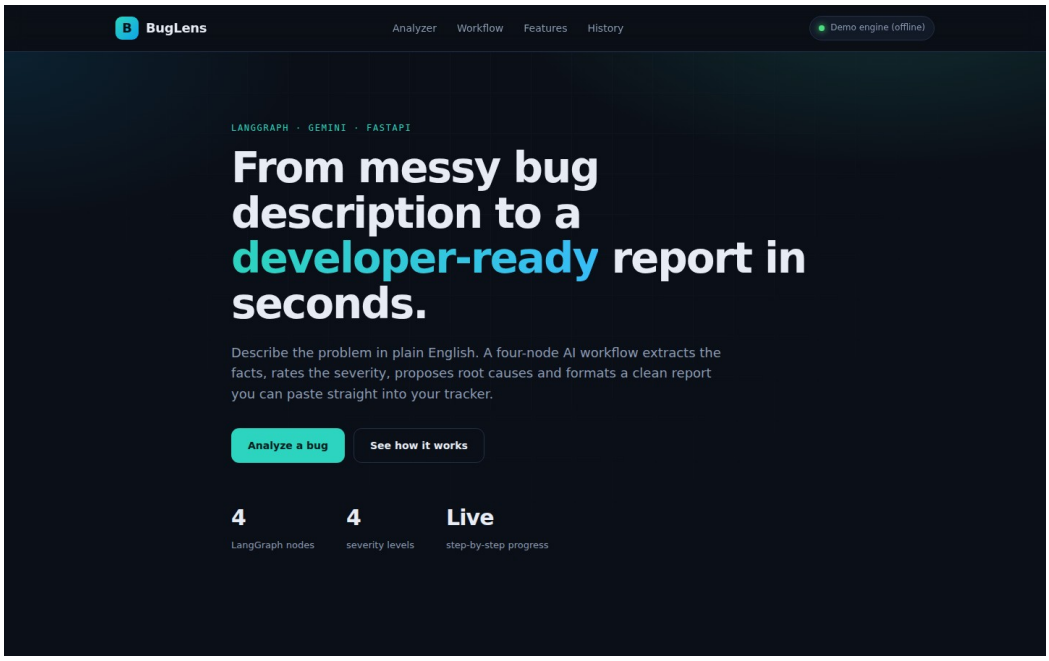


Figure 7.1 - Home page

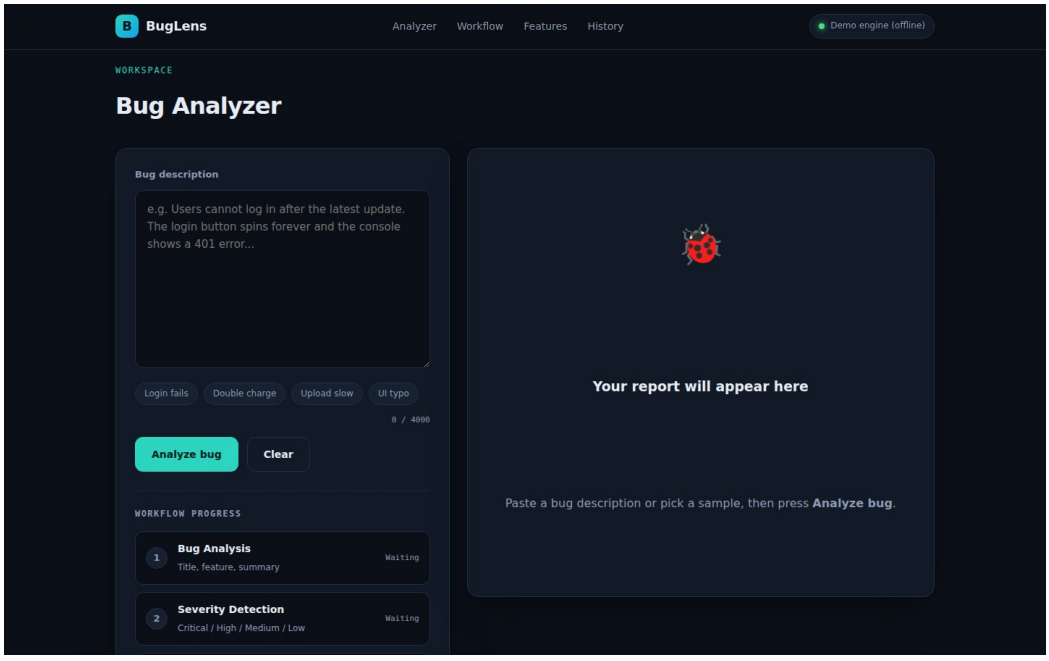


Figure 7.2 - Analyzer workspace before analysis

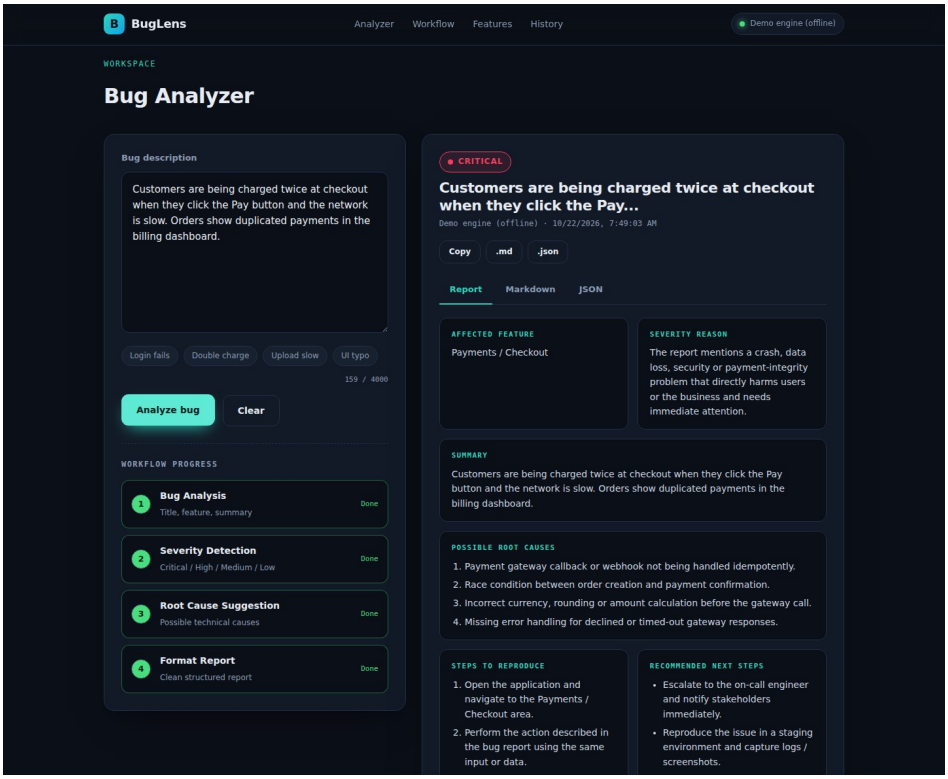


Figure 7.3 - Completed workflow and structured report

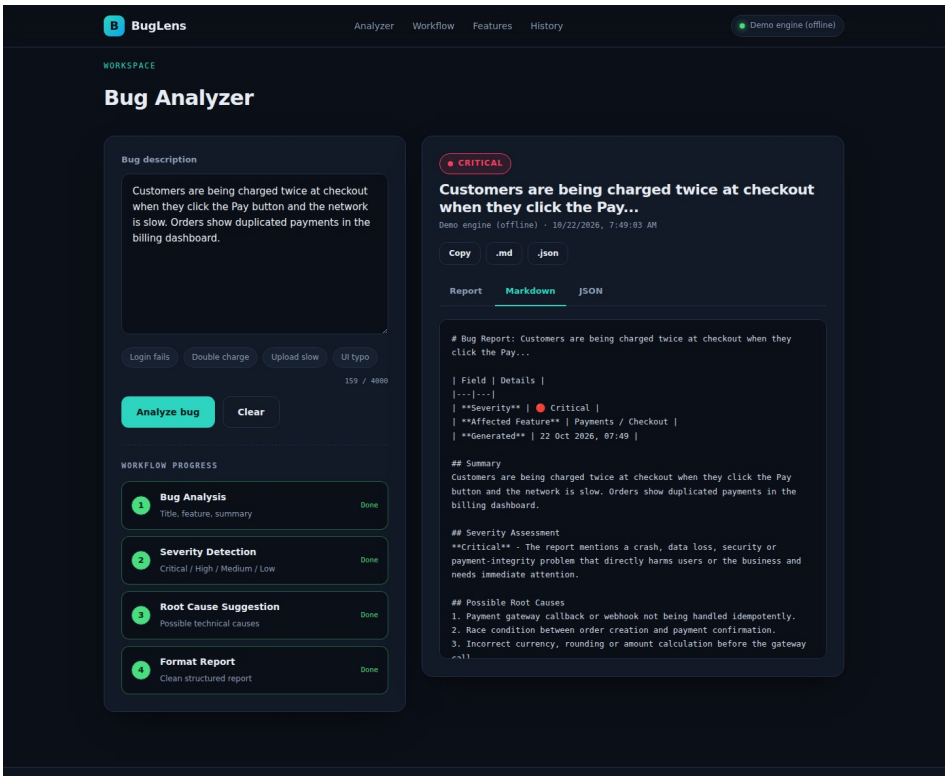


Figure 7.4 - Markdown export view

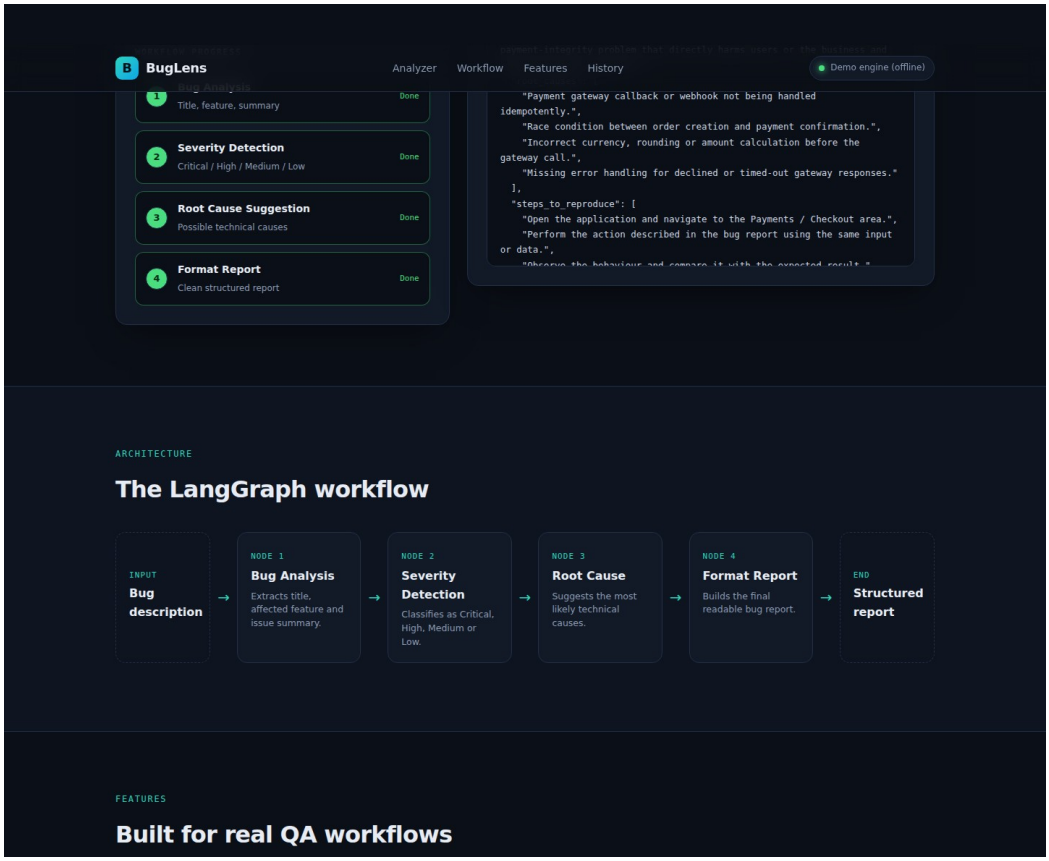


Figure 7.5 - Workflow architecture section

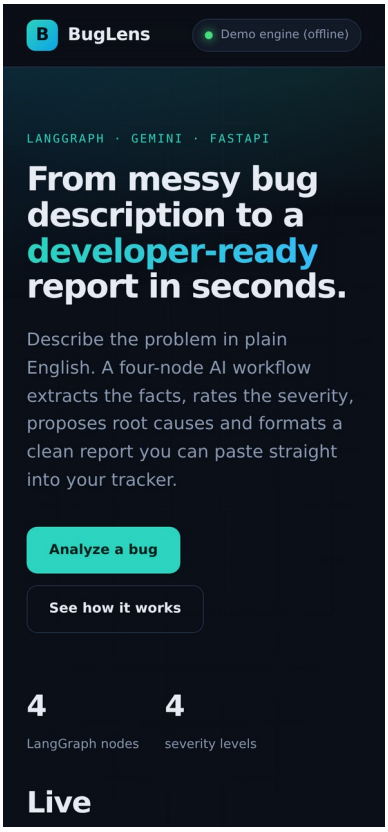


Figure 7.6 - Mobile layout

8. Testing and Validation

Automated tests are written with pytest and FastAPI's TestClient and run against the offline engine so they are deterministic.

Test	Checks	Result
test_graph_produces_full_state	Graph returns all state fields; crash with data loss is Critical	Pass
test_low_severity_for_cosmetic_bug	Spelling mistake is classified Low	Pass
test_api_health_and_analyze	Health and analyze endpoints respond correctly	Pass
test_validation_rejects_short_input	Too-short description returns HTTP 422	Pass

Manual testing covered all sample bugs, an empty input, a very long input, backend-offline behaviour and the mobile layout. Testing with a live Gemini key depends on the user's own API credentials and should be repeated after configuring .env.

9. Sample Input and Output

9.1 Input

> *Customers are being charged twice at checkout when they click the Pay button and the network is slow. Orders show duplicated payments in the billing dashboard.*

9.2 Generated Report (excerpt)

Field	Value
Affected feature	Payments / Checkout
Severity	Critical
Root cause 1	Payment gateway callback or webhook not being handled idempotently.
Root cause 2	Race condition between order creation and payment confirmation.
Next step	Escalate to the on-call engineer and reproduce the issue in staging.

This example was produced by the offline engine. With Gemini enabled the wording of the title, summary and causes is generated by the model and is usually more specific to the description.

10. Challenges and Solutions

Challenge	Solution
LLM output varies in format	Structured output with Pydantic schemas on every node
Application unusable without an API key	Offline rule-based fallback engine
Users see nothing while the workflow runs	Server-Sent Events with a live stepper
Keeping steps independent and testable	Typed shared state and partial updates per node
Inconsistent severity ratings	Explicit severity definitions inside the prompt

11. Limitations and Future Work

11.1 Limitations

- Root causes are hypotheses; they must be verified by an engineer.
- History is stored only in the user's browser.
- The fallback engine is keyword based and less accurate than Gemini.

11.2 Future Work

- Duplicate-bug detection using embeddings and a vector database.
- Export to Jira, GitHub Issues or Linear.
- Multimodal input: screenshots and log files.
- User accounts, a database and team analytics.
- Docker packaging and a CI/CD pipeline.

12. Conclusion

The AI Bug Report Analyzer meets every requirement of the assignment: it implements the four-node LangGraph workflow using Gemini and returns a clear structured bug report. Beyond the brief it adds streaming progress, structured output validation, exports, history, tests and documentation. The modular design makes it straightforward to add new nodes such as duplicate detection or ticket creation. The project shows how LangGraph can turn a single vague prompt into a reliable, explainable multi-step AI pipeline.

References

[1] LangGraph documentation - https://langchain-ai.github.io/langgraph/

[2] Google AI Studio and Gemini API - https://ai.google.dev/

[3] LangChain Google GenAI integration - https://python.langchain.com/docs/integrations/chat/google_generative_ai/

[4] FastAPI documentation - https://fastapi.tiangolo.com/

[5] Pydantic documentation - https://docs.pydantic.dev/

[6] MDN Web Docs: Server-sent events - https://developer.mozilla.org/docs/Web/API/Server-sent_events